

Momentum Flow 训练大纲进阶版：VAE Latent 与 MMDiT 主干

本文给出 Momentum Flow 的进阶主线训练大纲。相对基础版像素空间 TinyDiT 方案，本版引入两个关键变化：第一，图像不再直接作为 RGB 像素状态进入 Momentum Flow，而是先通过冻结的 SD3 风格 VAE encoder 映射到 latent 空间；第二，训练网络不再使用简化 TinyDiT，而是使用 SD3-inspired 的 MMDiT-lite 双流 Transformer 主干。后续所有高质量专项实验应优先基于本文进阶大纲，只修改数据规模、prompt 数量、模型大小、 τ 设置、loss 权重或采样器等局部变量，而不是重新定义一套训练框架。

该框架不是参考 Flow Matching 代码的直接改写，而是根据 Momentum Flow 导读与改进方案重新搭建的研究级训练代码。当前进阶版本仍然训练两个核心网络：端点方向网络 r_θ 与 velocity-space score 网络 $s_{\theta v}$ ；但二者的输入输出均定义在 VAE latent 空间，并使用 MMDiT-lite 主干处理 latent image stream 与 CLIP text stream。

1 训练目标

Momentum Flow 的核心思想是将普通 flow model 的位置状态 x_t 扩展为相空间状态

$$y_t = (x_t, v_t),$$

其中 x_t 是图像中间状态， v_t 是显式速度变量。给定真实图像 x_0 与噪声端点 ϵ ，定义端点方向

$$r = \epsilon - x_0.$$

在本进阶版实现中，Momentum Flow 的位置变量不再是 RGB 像素，而是 VAE latent。也就是说，训练代码先计算

$$z_0 = \text{VAEEnc}(x_0),$$

然后把基础公式中的位置变量 x_t 替换为 latent 变量 z_t ：

$$y_t = (z_t, v_t), \quad r = \epsilon - z_0.$$

为保持与 Momentum Flow 原始公式一致，后文推导中有时仍写 x_t 表示一般位置变量；落到进阶代码实现时，应读作 latent state z_t 。

正向条件 kinetic flow 写为

$$\begin{cases} dx_t = v_t dt, \\ dv_t = -\lambda(t)v_t dt + \beta(t)r dt + \sigma_\tau(t)dW_t. \end{cases}$$

训练好的两个网络用于反向采样。令反向时间 $m = 1 - t$, 反向状态

$$\tilde{y}_m = (\tilde{x}_m, \tilde{v}_m).$$

代码中使用带 `eta` 的 reverse kinetic ODE/SDE 统一形式:

$$d\tilde{x}_m = -\tilde{v}_m dm,$$

$$d\tilde{v}_m = \left[\lambda(t)\tilde{v}_m - \beta(t)r_\theta(\tilde{y}_m, t, \tau, e_p) + \frac{1 + \eta^2}{2}\sigma_\tau(t)^2 s_{\theta_v}(\tilde{y}_m, t, \tau, e_p) \right] dm + \eta\sigma_\tau(t)d\bar{W}_m.$$

其中 `eta=0` 时噪声项消失, 得到 deterministic probability-flow ODE; `eta=1` 时得到完整 stochastic reverse SDE。两个训练网络 r_θ 与 s_{θ_v} 正是在这个反向动力学中被调用。

训练阶段已知 x_0 与 ϵ , 因此可以构造条件训练样本 (x_t, v_t) , 并得到两个监督信号:

- 端点方向 $r = \epsilon - x_0$;
- 条件 velocity score $s_{\text{cond},v}$ 。

因此主线模型训练两个网络:

$$r_\theta(y_t, t, \tau, c) \approx \mathbb{E}[r \mid y_t, t, \tau, c],$$

$$s_{\theta_v}(y_t, t, \tau, c) \approx \nabla_v \log q_t(y_t \mid c).$$

这里 c 表示条件信息。当前主线实验中 c 来自冻结预训练 CLIP text encoder embedding; 代码仍保留 prompt id embedding 作为可选消融。

2 数据格式

主线代码将训练数据和生成数据分开。训练数据包含图像和 prompt; 生成数据只包含 prompt 文本。

2.1 训练数据格式

训练集放在 `Data/train_dataset/` 下, 但不直接把所有数据混在该目录根部。每个下载或整理后的数据集应单独放入一个数据集名称目录, 例如 `imagenet/`、`celeba/`、`cifar/` 或 `example_prompt_image/`。当前基础代码推荐统一使用一个 prompt 对一张图片的 manifest 格式:

```
Data/train_dataset/
  example_prompt_image/
    samples.json
    images/
      prompt_00.png
      prompt_01.png
  imagenet/
    samples.json
```

```
images/
celeba/
samples.json
images/
```

`samples.json` 记录每张图像与 prompt 文本之间的一一对应关系。其中 `image` 路径相对于该数据集目录，例如相对于 `Data/train_dataset/example_prompt_image/`：

```
[
  {"image": "images/prompt_00.png", "prompt": "a red car on a road"},
  {"image": "images/prompt_01.png", "prompt": "a blue vase on a table"}
]
```

训练数据读取后返回

$$(x_0, p),$$

其中 x_0 是真实图像， p 是该图像对应的 prompt 文本。

2.2 生成数据格式

生成或采样用 prompt 放在 `Data/sample_dataset/`。同样需要按 prompt set 名称再分一层，便于区分不同测评 prompt 集。该数据集不包含图像，只包含待生成的文本条件：

```
Data/sample_dataset/
example_prompts/
  prompts.json
imagenet_prompts/
  prompts.json
celeba_prompts/
  prompts.json
```

`prompts.json` 可以是 prompt 字符串列表：

```
[
  "a red car on a road",
  "a blue vase on a table"
]
```

生成数据读取后只返回

$$p,$$

其中 p 是生成条件文本。

3 冻结模型：VAE 与 CLIP

进阶版训练框架使用两个冻结预训练模型，它们不参与梯度更新，只作为固定的数据变换模块。本节详细说明两个模型的内部参数、输入输出尺寸和功能。

3.1 VAE：图像与 latent 之间的桥梁

VAE (Variational Autoencoder) 的作用是把高分辨率 RGB 图像压缩到低维 latent 空间, 以及从 latent 空间还原回 RGB 图像。进阶版使用 SD3 medium VAE, 从 HuggingFace `diffusers` 加载, 所有参数冻结。

VAE 的内部参数 VAE 的压缩能力由两个内置参数决定, 代码从模型 config 中自动读取, 不由用户手工设置:

- `block_out_channels`: 编码器/解码器中每个下采样 block 的输出通道数。SD3 medium VAE 为 (128, 256, 512, 512), 共 4 层;
- `latent_channels`: 最终 latent 空间的通道数。SD3 medium VAE 为 $C_z = 16$ 。

由 `block_out_channels` 的长度可自动推算出空间下采样倍率 (`vae_scale_factor`):

$$f = 2^{\text{len}(\text{block_out_channels})-1} = 2^{4-1} = 8.$$

即空间每边压缩 8 倍。除压缩比外, VAE 内部还有两个用于数值缩放的内置参数:

- `scaling_factor`: 将 latent 分布缩放到更适合扩散/流模型训练的尺度 (SD3 VAE 约为 0.13025);
- `shift_factor`: latent 分布的均值平移量 (SD3 VAE 约为 0.0)。

编码 (RGB $\rightarrow$ latent) 训练时, 每张真实图像 x_0 先经过冻结 VAE encoder 映射到 latent 空间:

$$z_0 = \text{VAEEnc}(x_0).$$

内部流程为:

$$x_0 (B, 3, 256, 256) \xrightarrow{\text{encoder}} z_0 (B, 16, 32, 32).$$

其中取 mode (μ) 而非从 $\mathcal{N}(\mu, \sigma^2)$ 采样, 保证同一张图每次编码结果确定不变。最终输出的 z_0 形状为 $(B, 16, 32, 32)$, 即 latent 通道数 $C_z = 16$, 空间尺寸 $H_z = W_z = 256/8 = 32$ 。

解码 (latent $\rightarrow$ RGB) 生成阶段, reverse kinetic 采样器输出的 $\hat{z}_0$ 经过冻结 VAE decoder 还原为 RGB 图像:

$$\hat{x}_0 = \text{VAEDec}(\hat{z}_0).$$

内部流程为:

$$\hat{z}_0 (B, 16, 32, 32) \xrightarrow{\text{反缩放}} \xrightarrow{\text{decoder}} \hat{x}_0 (B, 3, 256, 256).$$

解码后图像 clamp 到 $[-1, 1]$ 范围。这是因为训练数据本身归一化在 $[-1, 1]$ 内, clamp 仅作为安全措施, 防止 decoder 因数值误差产生略微越界的像素值 (如 -1.002), 正常解码几乎不会触发。

冻结说明 VAE 的所有参数(encoder + decoder 的卷积权重、scaling_factor、shift_factor) 均不参与训练。训练时的梯度只流经 z_0 之后的 Momentum Flow 状态和 MMdIT 网络，不回流到 VAE。代码通过 `requires_grad_(False)` 和 `torch.no_grad()` 双重保证。

3.2 CLIP：文本条件的编码器

CLIP text encoder 的作用是把 prompt 文本转化为神经网络可消费的条件向量。进阶版使用 CLIP-L/14 (`openai/clip-vit-large-patch14`)，从 HuggingFace `transformers` 加载，所有参数冻结。

输入：prompt $\rightarrow$ token ids prompt 文本首先经过 CLIP tokenizer：

$$p(\text{字符串}) \xrightarrow{\text{tokenizer}} \text{token_ids} \in \mathbb{R}^{B \times N_{\text{text}}},$$

其中 $N_{\text{text}} \leq 77$ (CLIP 默认最大 token 长度，自动 padding 和截断)。

输出：pooled embedding 与 token sequence token ids 送入冻结的 CLIP text model：

$$\text{CLIPTextModel}(\text{token_ids}) \rightarrow \begin{cases} e_p = \text{pooler_output} \in \mathbb{R}^{B \times 768}, \\ C_{\text{raw}} = \text{last_hidden_state} \in \mathbb{R}^{B \times N_{\text{text}} \times 768}. \end{cases}$$

两个输出的用途不同：

- e_p (pooled embedding)：整句 prompt 的全局语义向量，与时间 t 、预算 τ 的正弦编码一起进入 MLP，得到全局调制条件 $y \in \mathbb{R}^{B \times D}$ ；
- C_{raw} (token sequence)：每个 token 位置上的 hidden state，保留了词级别的细粒度信息。

维度投影 C_{raw} 的最后一维是 CLIP 的 hidden size (768)，与 MMdIT 的内部维度 D (默认 512) 不一致。因此 C_{raw} 在进入 MMdIT 前先经过一个可训练的线性投影层 W_c ：

$$C_0 = W_c(C_{\text{raw}}) \in \mathbb{R}^{B \times N_{\text{text}} \times D}.$$

W_c 是 MMdIT 的一部分，随网络一起训练。而 CLIP text model 本身的参数完全冻结。

冻结说明 CLIP text encoder 和 tokenizer 的所有参数均不参与训练，仅 W_c 投影层可训练。这样在对比实验中只改变生成模型或数据规模，文本条件机制的正规性不受影响。

训练与生成共用 训练和生成阶段使用完全相同的 CLIP 加载方式和参数，确保文本条件在训练和推理中一致。代码中 `Training/text_encoder.py` 和 `Sampling/text_encoder.py` 各自保留一份独立副本，使采样模块可以不依赖训练目录独立运行。

4 训练模型：MMDiT

前两节说明了冻结 VAE 如何将 RGB 图像压缩为 latent z_0 ，以及冻结 CLIP 如何将 prompt 文本编码为 e_p 和 C_0 。本节在此基础上讲解可训练的核心网络——MomentumMMDiT (SD3-inspired MMDiT-lite)，它接收 latent 相空间状态和文本条件，输出端点方向预测 $\hat{r}$ 或 velocity score 预测 $\hat{s}_v$ 。两个网络 r_θ 与 s_{θ_v} 结构完全相同但参数独立，以下以单个 MomentumMMDiT 为例说明。

4.1 从 VAE latent 和 CLIP 到 MMDiT 输入

回顾前两节的数据 size：

- VAE latent z_t, v_t 各为 $(B, 16, 32, 32)$ ($C_z = 16, H_z = W_z = 32$)；
- CLIP pooled embedding e_p 为 $(B, 768)$ ；
- CLIP token sequence C_{raw} 为 $(B, N_{\text{text}}, 768)$ 。

MMDiT 接收的输入为相空间状态 $\text{concat } u_t = \text{concat}(z_t, v_t) \in \mathbb{R}^{B \times 32 \times 32 \times 32}$ ($2C_z = 32$)、时间 t 、预算 τ 、 e_p 和 C_{raw} 。

4.2 Patch Embedding: latent 状态 $\rightarrow$ image stream

首先对 u_t 做 patchify。设 patch size $p = 2$ ，latent 网格 32×32 被划分为 $(32/2) \times (32/2) = 16 \times 16 = 256$ 个 patch。每个 patch 包含 $2C_z \times p \times p = 32 \times 4 = 128$ 个值：

$$\text{Patch}(u_t) : \mathbb{R}^{B \times 32 \times 32 \times 32} \rightarrow \mathbb{R}^{B \times 256 \times 128}.$$

再经线性投影 W_x 映射到 hidden dim D ，并加上可学习位置编码 P ：

$$X_0 = \text{Patch}(u_t) W_x + P \in \mathbb{R}^{B \times 256 \times D}, \quad D = 768.$$

X_0 即为 image stream，共 256 个 image tokens。

4.3 Text Projection: CLIP token $\rightarrow$ text stream

CLIP 输出的 $C_{\text{raw}} \in \mathbb{R}^{B \times N_{\text{text}} \times 768}$ 经过可训练线性层 W_c 投影到同一个 hidden dim D ：

$$C_0 = C_{\text{raw}} W_c \in \mathbb{R}^{B \times N_{\text{text}} \times D}, \quad D = 768.$$

C_0 即为 text stream，共 N_{text} 个 text tokens (≤ 77)。

4.4 全局调制条件

时间 t 和预算 τ 经正弦编码 + MLP 得到时间嵌入，再与 CLIP pooled embedding $e_p \in \mathbb{R}^{B \times 768}$ 的线性投影相加：

$$y = \text{MLP}(\text{Sin}(t, \tau)) + W_p e_p \in \mathbb{R}^{B \times D}.$$

y 作为全局调制条件，注入到每一个 MMDiT block 的 AdaLN 调制中。

4.5 MMDiT Block: 双流 Joint Attention

MMDiT 的核心是一个个串联的 MMDiT block。每个 block 内部，image stream 和 text stream 各自使用独立的 LayerNorm + QKV 参数，然后在 attention 中拼接：

1. **调制**：image stream 和 text stream 分别经 AdaLN 调制（shift/scale 由 y 产生）；
2. **独立 QKV**：image stream 用 W_Q^x, W_K^x, W_V^x 计算 Q_x, K_x, V_x ；text stream 用 W_Q^c, W_K^c, W_V^c 计算 Q_c, K_c, V_c ；
3. **拼接**： $Q = [Q_c; Q_x]$, $K = [K_c; K_x]$, $V = [V_c; V_x]$ ，沿 token 维拼接，text tokens 在前、image tokens 在后；

4. **Joint Attention**：

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V,$$

其中 $d_h = D/\text{heads}$ 为每个 head 的维度；

5. **拆分**：将 O 按 text 和 image 的 token 数拆回 O_c 和 O_x ；
6. **输出 + FFN**：各自经 output 投影和门控残差连接，再经独立的 SwiGLU MLP 更新。

形式上，第 ℓ 个 block 的更新可写为：

$$(X_{\ell+1}, C_{\ell+1}) = \text{MMDiTBlock}_\ell(X_\ell, C_\ell, y).$$

4.6 最终输出： $\hat{r}$ 与 $\hat{s}_v$

经过 L 个 block 后，只取 image stream 的最终状态 X_L ，经 AdaLN 调制和输出投影，unpatchify 回 latent 空间形状。整个过程输入与输出同尺寸：

$$(z_t, v_t, t, \tau, e_p, C_0) \xrightarrow{\text{MMDiT}} \begin{cases} \hat{r} \in \mathbb{R}^{B \times C_z \times H_z \times W_z}, & \text{对 } r_\theta \text{ 网络,} \\ \hat{s}_v \in \mathbb{R}^{B \times C_z \times H_z \times W_z}, & \text{对 } s_{\theta_v} \text{ 网络.} \end{cases}$$

两个网络的输出形状完全相同（均为 $C_z \times H_z \times W_z$ ），但物理含义不同： $\hat{r}$ 是 latent 空间端点方向预测，直接参与 reverse SDE 的 drift 项 $\beta(t)r_\theta$ ； $\hat{s}_v$ 是 velocity-space score，进入 Anderson 修正项 $\frac{1+\eta^2}{2}\sigma_\tau^2 s_{\theta_v}$ 。两者在 reverse kinetic 动力学中各司其职，缺一不可。

参数 `in_channels` 和 `out_channels` 均由 VAE 自动推断（ $= 2C_z$ 和 C_z ），不在 config 中手工设置。`image_size` 同样由 VAE 推断为 H_z 。

推荐参数配置 以下参数以生成质量为首要目标，不因算力妥协。

参数	推荐值	作用于 4.1–4.6 中的
dim	768	所有 hidden 表示的宽度 D (4.2 W_x 、4.3 W_c 、4.4 MLP、4.5 QKV/FFN)
depth	24	Block 数量 L (4.5 串联层数; r_θ 和 s_{θ_v} 各 24 层)
heads	12	每层 head 数 (4.5 中 $d_h = D/\text{heads} = 64$)
patch_size	2	latent patch 边长 (4.2 中 $N_x = (H_z/p)^2$)
mlp_ratio	4.0	FFN 隐藏宽度 = $D \times \text{ratio}$ (4.5 SwiGLU 维度)
dropout	0.0	attention dropout (4.5 训练前期关闭)

表 1: MMDiT 推荐参数及其在推导中的对应量。

上表中 **dim** 是最核心的参数——它直接决定 4.2 中 W_x 的输出宽度、4.3 中 W_c 的目标维度、4.4 中调制向量 y 的维度、以及 4.5 中所有 QKV 和 FFN 层的宽度。选定 $D = 768$ 与 CLIP-L/14 的原生 hidden size 对齐, text token 投影时不做维度压缩, 保留完整语义信息。**depth** 控制 4.5 中 MMDiT block 的串联数量, 不改变任何 tensor shape, 但决定了网络的表达深度。**heads** 控制 4.5 中 multi-head attention 的并行度, head 维 $d_h = D/\text{heads} = 64$ 是常用配置。**patch_size** 控制 4.2 中 token 数 N_x , $p = 2$ 时 32×32 latent 产生 256 个 image tokens。**mlp_ratio** 控制 4.5 中 SwiGLU FFN 的隐藏层宽度 $D \times 4.0 = 3072$ 。

在此配置下, 单个 MomentumMMDiT 约 370M 参数, 两个网络共约 740M 可训练参数。对于 256×256 RGB (latent 32×32), image tokens $N_x = 256$, text tokens $N_{\text{text}} \leq 77$, joint attention 总 token 量约 333。若后续分辨率提升至 512×512 (latent 64×64 , $N_x = 1024$), 建议 **dim=1024**、**depth=32**。

相比 TinyDiT 的优势 原始版本使用的 TinyDiT 仅接收 pooled text embedding e_p 作为 AdaLN 调制条件, image tokens 做纯自注意力, text 信息只通过一个全局向量注入。MMDiT 的改进体现在三处: (1) **token-level 条件**——保留完整 token sequence C_0 , 每个 image patch 在 joint attention 中可以直接从任意 text token 读取词级语义, 对多对象、空间关系等复杂 prompt 的对齐能力远强于单一 pooled vector; (2) **双流参数分离**——image stream 和 text stream 各自使用独立的 QKV 和 MLP 权重, 避免两种模态在 attention 空间中相互干扰; (3) **joint attention 通路**——text tokens 和 image tokens 在同一 attention 中直接交互, 信息通路比 AdaLN-only modulation 更直接、更丰富。

5 模型采样

下面以一张具体例子走通采样全流程: 假设我们已有训练好的 r_θ 和 s_{θ_v} checkpoint, 生成 prompt 为 “a cat sitting on a sofa” (7 个 token, 不足 77 则自动 padding), 输入 RGB 尺寸为 256×256 (VAE 压缩后 latent 为 $32 \times 32 \times 16$)。采样参数取 $\text{steps} = 100$, $\tau = 0.1$, $\text{eta} = 1$ (全随机 SDE)。

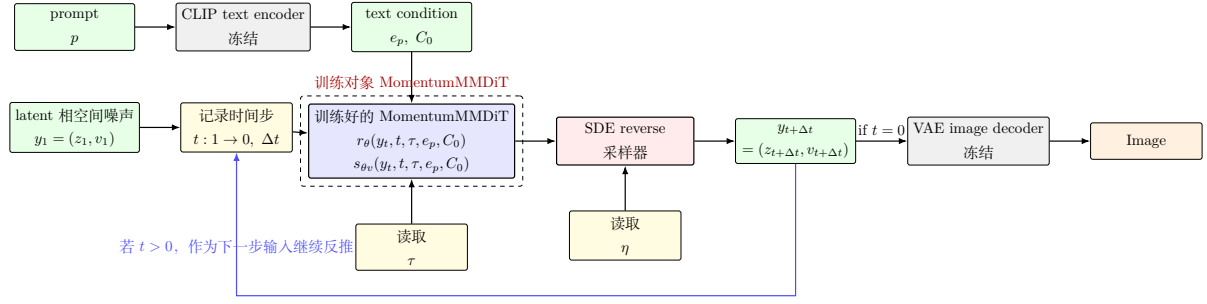


图 1: Momentum Flow 采样器的数据加工流程

Step 1 ——加载 checkpoint 与重建网络 代码入口为 `Sampling/generation.py` 的 `load_generator(ckpt_path)`。它从 checkpoint 中读取训练时保存的完整 config、 r_θ 权重和 $s_{\theta v}$ 权重，按 config 中的 `model.name=momentum_mmdit`、`dim=768`、`depth=24` 等参数重建两个 MomentumMMDiT 网络，同时加载冻结 CLIP text encoder (`Sampling/text_encoder.py`) 和冻结 VAE decoder (`Sampling/latent_vae.py`)。注意 `Sampling/` 目录自带这些冻结模块的独立副本，不依赖 `Training/`。

Step 2 ——编码 prompt prompt “a cat sitting on a sofa” 经 CLIP tokenizer 变为 token ids，形状 $(B, N_{\text{text}}) = (1, 7)$ (padding 后补齐到 77，但 attention mask 忽略 padding 位)。送入冻结 CLIP text model，得到：

$$e_p \in \mathbb{R}^{1 \times 768}, \quad C_0 \in \mathbb{R}^{1 \times N_{\text{text}} \times 768}.$$

e_p 经 W_p 投影参与全局调制， C_0 经 W_c 投影后进入 MMDiT 的 text stream（第 4 章已推导，此处不再展开）。

Step 3 ——初始化 latent 相空间 采样器在 latent 空间（而非 RGB 像素空间）初始化相空间状态。代码 `Sampling/reverse.py` 的 `sample_reverse_kinetic` 负责此步：

$$z_1 \sim \mathcal{N}(0, I) \in \mathbb{R}^{1 \times 16 \times 32 \times 32}, \quad v_1 \sim \mathcal{N}(0, Q_1 I) \in \mathbb{R}^{1 \times 16 \times 32 \times 32}.$$

Q_1 由训练时同一套 kinetic schedule (`Sampling/schedules.py`) 在 $t = 1$ 处计算。此时没有任何真实图像参与——生成过程只从噪声 prior、prompt 和训练好的网络出发。

Step 4 ——反向时间迭代 采样器在 $t: 1 \rightarrow 0$ 上均匀取 100 步 ($t_k = 1 - k/100$)，每步执行：

1. 将当前状态 $y_t = (z_t, v_t)$ 、当前 t 、 $\tau = 0.1$ 以及 Step 2 的 e_p, C_0 送入 r_θ ，得到 $\hat{r} \in \mathbb{R}^{1 \times 16 \times 32 \times 32}$ ；
2. 同样输入送入 $s_{\theta v}$ ，得到 $\hat{s}_v \in \mathbb{R}^{1 \times 16 \times 32 \times 32}$ ；
3. 按 reverse kinetic ODE/SDE 更新：

$$z_{t+\Delta t} = z_t + \Delta t \cdot v_t, \quad v_{t+\Delta t} = v_t + \Delta t \cdot \left(\beta \hat{r} - \lambda v_t - \frac{1+\eta^2}{2} \sigma^2 \hat{s}_v \right) + \eta \sigma \sqrt{-\Delta t} \varepsilon,$$

其中 $\Delta t = -1/100 < 0$ （反向）， $\varepsilon \sim \mathcal{N}(0, I)$ 。eta = 1 时噪声项生效，为全随机 SDE 采样。

100 步迭代完成后得到 $\hat{z}_0 \in \mathbb{R}^{1 \times 16 \times 32 \times 32}$ 。

Step 5 ——VAE decode 输出图像 将 $\hat{z}_0$ 送入冻结 VAE decoder (Sampling/latent_vae.py 的 FrozenLatentVAE.decode):

$$\hat{x}_0 = \text{VAEDec}(\hat{z}_0) \in \mathbb{R}^{1 \times 3 \times 256 \times 256}.$$

输出 clamp 到 $[-1, 1]$ 后映射到 $[0, 255]$ 像素值, 保存为 PNG。

代码调用小结 整个采样流程由 Sampling/generation.py 的 generate_samples_from_checkpoint 一行调用完成, 内部依次调用 load_generator (加载 checkpoint + 重建网络)、encode_generation_prompts(编码 prompt)、sample_reverse_kinetic(Sampling/reverse.py, 反向积分)、VAEDecode (Sampling/latent_vae.py)。后续测评代码应直接调用该接口, 不复制采样逻辑。

6 默认训练参数

本节列出当前主线实验的默认参数配置, 对应代码文件 configs/base_training.py。模型结构 (dim, depth, heads 等)、冻结模块 (VAE, CLIP) 和 preview 启停已在第 3-5 章详述, 此处仅列出训练特有的可调参数。

cfg.dataset.image_size 不是 Transformer 看到的 latent 尺寸, 而是送入 VAE encoder 前统一 resize + center crop 后的 RGB 图像边长 (默认 256)。原始图片可以是混合分辨率, 但进入 VAE 前会被整理成固定方形尺寸以保证 batch 内 token 网格一致。给定 dataset.image_size=256 和 VAE 下采样倍率 $f = 8$, 代码自动推断 $H_z = W_z = 32$, 进而设置 model.image_size=32、model.out_channels= $C_z = 16$ model.in_channels = $2C_z = 32$ 。只需保证 image_size 能被 f 整除且 H_z 能被 patch_size 整除。

表 2: 默认训练参数。

类别	参数	默认值
数据	<code>dataset.image_size</code>	256 (RGB 边长, 送入 VAE 前)
数据	<code>dataset.batch_size</code>	64
Kinetic	<code>kinetic.lambda_const</code>	2.0
Kinetic	<code>kinetic.rho</code>	2.0
Kinetic	<code>kinetic.tau_min</code>	0.05
Kinetic	<code>kinetic.tau_max</code>	0.20 (LogUniform 采样)
Kinetic	<code>kinetic.num_quad</code>	128
优化器	<code>optimizer_r.lr</code> / <code>optimizer_s.lr</code>	1×10^{-4}
优化器	<code>optimizer_r.betas</code> / <code>optimizer_s.betas</code>	(0.9, 0.95)
优化器	<code>optimizer_r.weight_decay</code> / <code>optimizer_s.weight_decay</code>	0.03
训练控制	<code>train.steps</code>	100000
训练控制	<code>train.grad_clip_norm</code>	1.0
训练控制	<code>train.log_every</code>	10
训练控制	<code>train.save_every</code>	1000
训练控制	<code>train.keep_last_checkpoints</code>	5

6.1 数据尺寸

`dataset.image_size=256` 是送入 VAE encoder 前的统一 RGB 边长。经 VAE ($f = 8, C_z = 16$) 编码后变为 latent ($B, 16, 32, 32$), 此即 MMDiT 实际处理的 tensor。`dataset.batch_size=64` 为每步训练样本数, 若数据集较小可减小或关闭 `drop_last`。

6.2 Kinetic 参数

Momentum Flow 的均值路径取 $c(t) = 2t - t^2$, 满足 $c(0) = 0, c(1) = 1, c'(1) = 0$, 使终点速度先验不依赖真实图像。阻尼系数 $\lambda = 2.0$ 控制速度衰减速率。速度方差预算沿时间分配由 $g(t) = t^\rho$ ($\rho = 2$) 决定, 大部分随机性集中在噪声端 ($t \approx 1$)。随机性预算 τ 在 $[0.05, 0.20]$ 按 LogUniform 采样, 训练时模型学习在不同噪声水平下工作; 调试阶段可将 `tau_min` 和 `tau_max` 设为相同值固定 τ 。`num_quad=128` 是协方差 S_t, C_t, Q_t 的数值积分点数。

6.3 优化器配置

r_θ 和 s_{θ_v} 各使用独立 AdamW optimizer, 参数相同: 学习率 1×10^{-4} , betas (0.9, 0.95), weight decay 0.03。每个 step 先 `loss_r.backward(retain_graph=True)` $\rightarrow$ clip $\rightarrow$ `optimizer_r.step()`, 再 `loss_s.backward()` $\rightarrow$ clip $\rightarrow$ `optimizer_s.step()`。两个网络的梯度天然隔离, 独立裁剪 (`grad_clip_norm=1.0`), 不互相污染。若需为 `score_net` 设置不同学习率, 单独修改 `optimizer_s.lr` 即可。

7 模型训练

本节以代码文件 `Training/train.py` 为主线，逐步描述一个训练 step 的完整数据流。后续对比实验只改 config 或调用公共模块，不重写训练逻辑。

Step 1 ——数据抽取：prompt 与图像 训练首先根据 config 中的 `dataset.folder` 和 `dataset.manifest` 读取一个 batch。路径应指向具体数据集目录，例如：

```
Data/train_dataset/example_prompt_image/samples.json
Data/train_dataset/imagenet/samples.json
Data/train_dataset/celeba_prompt_image/samples.json
```

每条训练样本包含一张图像和一个 prompt。数据读取代码位于：

```
Data/train_dataset/prompt_image.py
```

其中 `PromptImageDataset` 负责读取 manifest，`build_image_transform` 负责 resize、crop、tensor 化和归一化。

数据模块输出：

$$(x_0, p),$$

其中 $x_0 \in [-1, 1]^{3 \times H \times W}$ 是归一化后的真实图像， p 是 prompt 文本。进阶版训练入口随后调用冻结 VAE encoder：

$$z_0 = \text{VAEEnc}(x_0),$$

并把 z_0 作为 Momentum Flow 的位置端点输入后续公共采样模块。

Step 2 ——文本条件编码 prompt 文本不直接进入 MomentumMMDiT。对应代码文件是 `Training/text_encoder.py`。该文件负责 tokenizer、冻结预训练 text model，并把 batch prompt 编码为 pooled embedding 与 token sequence：

$$e_p = \text{CLIPPooled}(p), \quad C_0 = \text{CLIPTokens}(p).$$

CLIP text encoder 只负责生成固定 text 条件，不参与 optimizer 更新。这样实验缩小的是生成模型和 prompt 数量，而不是削弱文本条件机制。

这里的两个 CLIP 结果来自同一个冻结 CLIP text encoder。 C_0 保留 token 维度，所以 MMDiT 可以在 attention 里读取每个词或子词的条件信息； e_p 把整句 prompt 压成一个全局向量，所以适合与 t, τ 一起作为整体调制条件。前者进入 text stream，后者进入 modulation MLP，二者分工不同但来源相同。

训练网络同时接收 pooled embedding e_p 与 text token stream C_0 。 e_p 用于 timestep/tau 的全局 modulation， C_0 进入 MMDiT 的 text stream，与 latent image stream 在 attention 中融合。

Step 3 ——公共随机量与条件 Gaussian flow 采样 两个 loss 共用同一批公共随机量和同一个训练状态。给定 (z_0, p) 后，公共采样模块完成以下计算。对应代码文件是 `Training/state_sampling.py`，其中应集中产生训练中公共使用的 $\epsilon, t, \tau, \mu_z, \mu_v, S_t, C_t, Q_t, y_t = (z_t, v_t)$ ，避免两个 loss 各自重复采样。

`Training/state_sampling.py` 是训练状态采样的主流程文件；`Training/schedules.py` 只是其中调用的数学工具文件，负责提供 $c(t), c'(t), g(t), \lambda(t), \sigma_\tau(t)$ 和协方差积分。也就是说，`schedules.py` 不直接决定一个 batch 要采什么，它只为 `state_sampling.py` 提供计算公式。

1. 采样噪声端点。由 `Training/state_sampling.py` 生成：

$$\epsilon \sim \mathcal{N}(0, I).$$

2. 采样时间。由 `Training/state_sampling.py` 根据 config 中的时间范围生成：

$$t \sim U(\epsilon_t, 1 - \epsilon_t).$$

3. 采样或设定随机性预算。由 `Training/state_sampling.py` 根据 config 中的 $\tau_{\min}, \tau_{\max}$ 生成：

$$\tau \sim \text{LogUniform}(\tau_{\min}, \tau_{\max}).$$

调试时可固定 $\tau_{\min} = \tau_{\max} = 0.05$ ；正式对比时可改为一个范围。

4. 计算端点方向标签。由 `Training/state_sampling.py` 直接得到，后续 `Training/loss.py` 会复用该 label：

$$r = \epsilon - z_0.$$

5. 根据均值路径。路径函数和导数由 `Training/schedules.py` 提供：

$$c(t) = 2t - t^2, \quad c'(t) = 2 - 2t$$

`Training/state_sampling.py` 调用该 schedule 计算条件均值：

$$\mu_z(t) = (1 - c(t))z_0 + c(t)\epsilon,$$

$$\mu_v(t) = c'(t)(\epsilon - z_0) = c'(t)r.$$

6. 通过 `Training/schedules.py` 数值积分得到协方差标量

$$S_t, \quad C_t, \quad Q_t.$$

其中 S_t 是 z_t 的条件方差， Q_t 是 v_t 的条件方差， C_t 是 z_t 与 v_t 的条件协方差。代码中先定义阻尼传播核和位置累积核：

$$\Phi(t, u) = \exp\left(-\int_u^t \lambda(a) da\right),$$

$$K(t, u) = \int_u^t \Phi(s, u) ds.$$

在当前默认常数阻尼 $\lambda(t) = \lambda$ 下,

$$\Phi(t, u) = \exp[-\lambda(t - u)], \quad K(t, u) = \frac{1 - \exp[-\lambda(t - u)]}{\lambda}.$$

噪声注入强度为

$$\sigma_\tau(u) = \sqrt{\tau} \sqrt{g'(u) + 2\lambda(u)g(u)}.$$

因此三个协方差标量为

$$\begin{aligned} Q_t &= \int_0^t \Phi(t, u)^2 \sigma_\tau(u)^2 du, \\ C_t &= \int_0^t K(t, u) \Phi(t, u) \sigma_\tau(u)^2 du, \\ S_t &= \int_0^t K(t, u)^2 \sigma_\tau(u)^2 du. \end{aligned}$$

直观上, Q_t 控制速度变量的随机扰动, S_t 控制位置变量的随机扰动, C_t 描述位置扰动和速度扰动之间的相关性。当前代码用梯形积分近似上述积分。

7. 从条件相空间高斯分布采样训练状态。由 `Training/state_sampling.py` 使用 $\mu_x, \mu_v, S_t, C_t, Q_t$ 共同采样:

$$y_t = (z_t, v_t) \sim \mathcal{N} \left(\begin{bmatrix} \mu_z(t) \\ \mu_v(t) \end{bmatrix}, \begin{bmatrix} S_t I & C_t I \\ C_t I & Q_t I \end{bmatrix} \right).$$

因此, 第一个 loss 和第二个 loss 使用同一个 $t, \tau, y_t = (z_t, v_t)$ 。这能保证两个监督目标在同一个训练状态上比较, 而不是各自重新采样。训练入口文件只应调用公共采样模块并接收这些结果, 不应在入口里手写上述公式。

这里的 z_t 与 v_t 是训练阶段从正向条件 Gaussian flow 的边缘分布中解析采样得到的当下 latent 状态, 不是通过反向 ODE/SDE 数值求解得到的。反向 ODE/SDE 用在生成阶段; 训练阶段这样采样, 是为了在任意时间 t 上直接构造监督 label。

Step 4 ——端点方向 loss: label、预测与梯度 端点方向 loss 的 label 很直接。对应代码文件是 `Training/loss.py`:

$$r_{\text{label}} = \epsilon - z_0.$$

网络输入由训练状态构成。这里 y_t 表示神经网络看到的相空间状态:

$$y_t = (z_t, v_t).$$

也就是说, z_t 提供当前 latent 位置状态, v_t 提供当前速度状态; t, τ, e_p 和 C_0 作为条件共同影响 Transformer 的输出。

端点方向网络为 MomentumMMDiT。网络结构定义在 `Model/networks/mmdit.py`, 训练入口把 y_t, t, τ, e_p, C_0 传给 `r_net`:

$$\hat{r} = r_\theta(y_t, t, \tau, e_p, C_0).$$

它的输入是当前状态 $y_t = (z_t, v_t)$ 、时间 t 、随机性预算 τ 、pooled 文本 embedding e_p 和 text token stream C_0 。输出形状为

$$\hat{r} \in \mathbb{R}^{C_z \times H_z \times W_z}.$$

端点方向网络使用未加权 MSE。紧凑地写，它的训练目标是

$$L_r = \mathbb{E}_{x_0, p, \epsilon, t, \tau, y_t} \left[\|r_\theta(y_t, t, \tau, e_p, C_0) - (\epsilon - z_0)\|_2^2 \right].$$

展开到一个 batch 中，对第 i 个样本，记像素和通道总索引为 j ，则 per-sample loss 为

$$\ell_{r,i} = \frac{1}{CHW} \sum_{j=1}^{CHW} (\hat{r}_{i,j} - r_{\text{label},i,j})^2.$$

训练时使用 batch 平均：

$$L_r = \frac{1}{B} \sum_{i=1}^B \ell_{r,i}.$$

也就是

$$L_r = \frac{1}{B} \sum_{i=1}^B \frac{1}{CHW} \sum_{j=1}^{CHW} (r_\theta(y_{t_i}, t_i, \tau_i, e_{p_i})_j - (\epsilon_i - x_{0,i})_j)^2.$$

该 loss 只更新 r_θ 的参数；CLIP text encoder 冻结，不参与梯度更新。训练入口只负责接收该 loss、反向传播和 optimizer step。

Step 5 —— Velocity score loss: label、预测与梯度 第二个 loss 使用同一个 $t, \tau, y_t = (z_t, v_t)$ ，但它的 label 更复杂，需要根据条件高斯解析计算。对应代码文件是 `Training/loss.py`。

公共采样模块已经给出：

$$x_t, \quad v_t, \quad \mu_x(t), \quad \mu_v(t), \quad S_t, \quad C_t, \quad Q_t.$$

其中：

- x_t ：从条件 Gaussian flow 采样得到的位置状态；
- v_t ：同一次采样得到的速度状态；
- $\mu_x(t)$ ：在给定 x_0, ϵ 下的位置条件均值；
- $\mu_v(t)$ ：在给定 x_0, ϵ 下的速度条件均值；
- S_t ： x_t 的条件方差标量；
- Q_t ： v_t 的条件方差标量；
- C_t ： x_t 与 v_t 的条件协方差标量。

条件高斯下 velocity score label 为：

$$s_{\text{cond},v} = \frac{C_t(x_t - \mu_x) - S_t(v_t - \mu_v)}{S_t Q_t - C_t^2}.$$

该 label 的计算放在 `Training/loss.py` 中, 避免散落在训练入口里。这样后续如果要替换 score 目标或做 score loss 消融, 只改 loss 模块。

这里需要区分两类随机性。采样 t 与 τ 更像是训练任务条件的随机性, 类似于 prompt 分布对 loss 的影响: 不同 t, τ 决定模型在这一 step 要学习哪个时间、哪个随机性预算下的局部任务。它们会改变 $\mu_x, \mu_v, S_t, C_t, Q_t$, 但一旦 t, τ, x_0, ϵ 已经确定, 这个样本的任务条件就已经确定, 此时

$$\mu_x, \mu_v, S_t, C_t, Q_t$$

都可以看作常数。接下来 velocity score label 仍然有随机性, 来源不是 t, τ , 而是训练状态采样中的 z_t, v_t 。

为看清该 label 的尺度, 固定 t, τ, x_0, ϵ , 并定义

$$X = x_t - \mu_x, \quad V = v_t - \mu_v.$$

条件 Gaussian flow 给出每个像素/通道维度上的联合分布

$$\begin{bmatrix} X \\ V \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} S_t & C_t \\ C_t & Q_t \end{bmatrix} \right).$$

因此

$$\text{Var}(X) = S_t, \quad \text{Var}(V) = Q_t, \quad \text{Cov}(X, V) = C_t.$$

score label 的分子记为

$$N = C_t X - S_t V.$$

由于 N 是联合高斯变量的线性组合, 它仍然服从高斯分布。其均值为

$$\mathbb{E}[N] = C_t \mathbb{E}[X] - S_t \mathbb{E}[V] = 0.$$

其方差为

$$\begin{aligned} \text{Var}(N) &= \text{Var}(C_t X - S_t V) \\ &= C_t^2 \text{Var}(X) + S_t^2 \text{Var}(V) - 2C_t S_t \text{Cov}(X, V) \\ &= C_t^2 S_t + S_t^2 Q_t - 2S_t C_t^2 \\ &= S_t(S_t Q_t - C_t^2). \end{aligned}$$

所以

$$N \sim \mathcal{N}(0, S_t(S_t Q_t - C_t^2)).$$

又因为

$$s_{\text{cond},v} = \frac{N}{S_t Q_t - C_t^2},$$

可得

$$s_{\text{cond},v} \sim \mathcal{N} \left(0, \frac{S_t}{S_t Q_t - C_t^2} \right) = \mathcal{N} \left(0, \frac{1}{Q_t - \frac{C_t^2}{S_t}} \right).$$

因此, 在固定 t, τ, z_0, ϵ 后, velocity score label 的有效尺度由

$$\sigma_{\text{eff}}^2(t, \tau) = Q_t - \frac{C_t^2}{S_t}$$

决定。它不是瞬时噪声注入强度 $\sigma_\tau(t)^2$ ，而是给定 z_t 后速度变量 v_t 的条件方差。

Velocity score 网络同样是 MomentumMMDiT，但参数与 r_θ 不共享。网络结构仍来自 `Model/networks/mmdit.py`，训练入口把同一个 y_t, t, τ, e_p, C_0 传给 `score_net`：

$$\hat{s}_v = s_{\theta_v}(y_t, t, \tau, e_p, C_0).$$

输出形状为：

$$\hat{s}_v \in \mathbb{R}^{C_z \times H_z \times W_z}.$$

方案一：原始 velocity score loss 最直接的 score 训练方案是不做尺度加权，直接使用原始 MSE：

$$L_s = \mathbb{E}_{x_0, p, \epsilon, t, \tau, y_t} \left[\|s_{\theta_v}(y_t, t, \tau, e_p) - s_{\text{cond}, v}\|_2^2 \right].$$

代码实现时再对 batch、通道和像素做平均。一步展开为

$$L_s = \frac{1}{B} \sum_{i=1}^B \frac{1}{CHW} \sum_{j=1}^{CHW} (s_{\theta_v}(y_{t_i}, t_i, \tau_i, e_{p_i})_j - s_{\text{cond}, v, i, j})^2.$$

这就是原始 Momentum Flow score matching 目标。

方案二：NCSN 风格加权 velocity score loss 若后续采用 NCSN 风格的尺度加权，目标不是改变 $s_{\text{cond}, v}$ 这个 label，而是让不同 t, τ 下的 score 误差进入 loss 时具有更接近的尺度。由前面的推导可知，在固定 t, τ, x_0, ϵ 后，

$$s_{\text{cond}, v} \mid t, \tau, x_0, \epsilon \sim \mathcal{N}\left(0, \frac{1}{\sigma_{\text{eff}}^2(t, \tau)}\right),$$

其中有效方差具体为

$$\sigma_{\text{eff}}^2(t, \tau) = Q_t - \frac{C_t^2}{S_t}.$$

因此 raw score label 的标准差为 $1/\sigma_{\text{eff}}(t, \tau)$ ；若希望不同 t, τ 下的 score 误差在 MSE 中具有接近的尺度，就应在平方误差前乘以有效方差。

未归一化的 NCSN 风格加权 loss 写成

$$L_s^{\text{NCSN}} = \mathbb{E}_{x_0, p, \epsilon, t, \tau, y_t} \left[\sigma_{\text{eff}}^2(t, \tau) \|s_{\theta_v}(y_t, t, \tau, e_p) - s_{\text{cond}, v}\|_2^2 \right].$$

一步展开为

$$L_s^{\text{NCSN}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{CHW} \sum_{j=1}^{CHW} w_i (s_{\theta_v}(y_{t_i}, t_i, \tau_i, e_{p_i})_j - s_{\text{cond}, v, i, j})^2,$$

其中

$$w_i = \sigma_{\text{eff}, i}^2 = Q_{t_i} - \frac{C_{t_i}^2}{S_{t_i}}.$$

实际实现中通常使用归一化权重保持 batch loss 的整体量级稳定。令

$$\bar{w}_i = \frac{w_i}{\frac{1}{B} \sum_{k=1}^B w_k},$$

则最终采用的稳定版加权 loss 可写为

$$L_s^{\text{weighted}} = \mathbb{E}_{x_0, p, \epsilon, t, \tau, y_t} \left[\bar{w}(t, \tau) \|s_{\theta v}(y_t, t, \tau, e_p) - s_{\text{cond}, v}\|_2^2 \right].$$

其 batch 展开形式为

$$L_s^{\text{weighted}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{CHW} \sum_{j=1}^{CHW} \bar{w}_i (s_{\theta v}(y_{t_i}, t_i, \tau_i, e_{p_i})_j - s_{\text{cond}, v, i, j})^2.$$

在训练时随机采样 t 与 τ 的框架下，方案二通常更合理：它把不同时间和随机性预算下的 score label 尺度拉到更接近的量级，减少某些 t, τ 因天然 loss 小而学习不足、某些 t, τ 因天然 loss 大而主导训练的问题。若使用加权版本，应在实验记录中明确它是 score loss balancing trick，而不是 Momentum Flow label 本身的改变。

8 WandB 与 checkpoint

训练脚本应记录以下指标：

- `loss`：总损失；
- `loss_r`：端点方向损失；
- `loss_s`：velocity score 损失；
- `grad_norm_r` 与 `grad_norm_s`：两个网络各自的梯度范数，用于观察训练是否出现明显爆炸；
- `preview/images`：训练中定期用少量当前 prompt 做轻量生成，只用于快速观察训练趋势。

其中 `loss` 记录是正常训练必须保留的监控项。训练中 `preview` 生成不是正式测评流程，不调用 `Sampling/generation.py` 的完整生成入口；它通过 `Training/preview_sampling.py` 在训练模块内部用当前 batch 的少量 prompt 和当前权重跑短步数 reverse kinetic ODE/SDE，并把图片写入 wandb。由于当前任务的 prompt 数量很少、类别相近，这种监控可以帮助快速判断模型是否已经开始学到基本方向。

WandB 不应把 API key 或服务器私有路径写死在代码中。训练配置只保存可公开的运行选项：`wandb.enabled` 控制是否启用，`wandb.project`、`wandb.entity`、`wandb.run_name`、`wandb.mode`、`wandb.tags` 和 `wandb.notes` 控制上传到哪个项目以及如何命名。当前基础配置可以默认指向本项目的 wandb 位置：

```
entity = MAIR_HUST
project = Momentum-Flow
```

这两个字段不是秘密，可以写进 config；真正不能写进代码的是 API key。

若 `wandb.mode=online`，运行前应在当前服务器用户下执行 `wandb login`，或在服务器环境变量中设置 `WANDB_API_KEY`。第一次登录需要输入 API key；登录成功后，该 key 会保存在当前服务器用户自己的 wandb 配置目录中，后续同一用户再次运行通常不需要重复输入。如果未登录，

训练入口应直接提示先登录，而不是静默失败。别人如果要把 run 上传到该 entity/project，应在 wandb 网页端被加入对应 team/entity；不要共享个人 API key。共享给他人的代码可以把 wandb.enabled 改为 false，或运行时使用 --no-wandb。

checkpoint 应保存：

- 当前 step；
- r_net 权重；
- score_net 权重；

当前 baseline checkpoint 保存两个训练网络的全权重、step 和训练 config。保存间隔由 train.save_every 控制（默认 1000 step）。为避免 checkpoint 目录无限增长，Training/train.py 默认只保留最近 train.keep_last_checkpoints=5 个 checkpoint。

9 当前代码文件职责

当前主线代码目录如下：

```
OurMomentumFlow/
|-- Data/
|   |-- train_dataset/
|   |   |-- prompt_image.py
|   |   |-- example_prompt_image/
|   |   |   |-- samples.json
|   |   |-- celeba_prompt_image/
|   |       |-- samples.json
|   |-- sample_dataset/
|       |-- prompts.py
|       |-- example_prompts/
|           |-- prompts.json
|       |-- ...
|-- Training/
|   |-- train.py
|   |-- text_encoder.py
|   |-- latent_vae.py
|   |-- schedules.py
|   |-- state_sampling.py
|   |-- loss.py
|   |-- preview_sampling.py
|-- Sampling/
|   |-- generation.py
|   |-- reverse.py
|   |-- text_encoder.py
|   |-- latent_vae.py
|   |-- schedules.py
```

```

|-- Model/
|   |-- networks/
|   |   |-- mmdit.py
|   |   |-- factory.py
|   |-- checkpoints/
|-- configs/
|   |-- loader.py
|   |-- base_training.py
|   |-- base_sampling.py
|   |-- exp1/
|       |-- part1_config.py
|-- experiments/
|   |-- exp1/
|       |-- part1.sh
|       |-- README.md

```

9.1 Data/

Data/train_dataset/ 保存训练数据读取代码和不同训练数据集目录。数据本体必须再按数据集名分层，例如 imagenet/、celeba_prompt_image/ 或 cifar/。其中 prompt_image.py 负责读取一个 prompt 对一张图片的 manifest 数据集；每个数据集目录中的 samples.json 记录图像相对路径和 prompt 文本。

Data/sample_dataset/ 保存生成或测评用 prompt set，也需要按名称分层，例如 Data/sample_dataset/example_prompts/。Data/sample_dataset/prompts.py 负责读取 prompts.json 或文本文件，并只返回 prompt 字符串列表，不读取训练图像。

9.2 Training/

Training/train.py 是训练入口。它从 configs/ 读取配置，构造 dataset、dataloader、冻结 text encoder、冻结 VAE、两个 MomentumMMDiT 网络、双 AdamW optimizer，并负责 loss 反向传播、梯度裁剪、optimizer step、wandb 日志和 checkpoint 保存。

Training/text_encoder.py 负责冻结 CLIP text encoder，输入 prompt 字符串，输出 pooled embedding e_p 、text token sequence C_0 和 attention mask。Training/latent_vae.py 负责冻结 VAE encoder/decoder，训练时把 RGB 编码为 latent z_0 ，preview 时把 latent 解码回 RGB。Training/schedules.py 提供纯数学函数： $c(t)$ 、 $c'(t)$ 、 $\sigma_\tau(t)$ 及协方差 S_t 、 C_t 、 Q_t 的数值积分。Training/state_sampling.py 负责采样公共训练状态： ϵ 、 t 、 τ 、 (z_t, v_t) 及 μ_z 、 μ_v 、 S_t 、 C_t 、 Q_t 。Training/loss.py 负责计算 L_r 和 L_s 及 velocity score label。Training/preview_sampling.py 仅用于训练中 wandb 预览，不是正式生成入口。

9.3 Model/

Model/networks/mmdit.py 实现 MomentumMMDiT 主干 (MMDiT-lite 双流 Transformer)；Model/networks/factory.py 根据 config 构造网络实例。训练阶段会创建两个结

构相同但参数不共享的网络： r_θ 和 $s_{\theta v}$ 。checkpoint 默认保存到 `Model/checkpoints/`，包含 step、训练 config、 r_θ 和 $s_{\theta v}$ 权重。

9.4 Sampling/

`Sampling/` 不依赖 `Training/` 目录，可独立完成生成。内部保留训练阶段部分模块的独立副本：

`Sampling/generation.py` 是生成编排层：加载 checkpoint、按训练 config 重建 r_θ 和 $s_{\theta v}$ 、加载冻结 text encoder 和 VAE decoder、编码 prompt、调用 reverse 积分器、VAE decode 输出图像。后续测评代码应直接调用该接口。`Sampling/reverse.py` 实现 reverse kinetic ODE/SDE 积分器，在 latent 空间中从 y_1 反推到 $\hat{z}_0$ 。`Sampling/text_encoder.py` 为冻结 CLIP text encoder 独立副本；`Sampling/latent_vae.py` 为冻结 VAE decoder 独立副本；`Sampling/schedules.py` 为 kinetic schedule 数学工具独立副本。

9.5 experiments/

`experiments/` 是实验入口目录。组织规则是：实验编号用文件夹表示，例如 `experiments/exp1/`、`experiments/exp2/`；同一实验下的不同阶段用文件表示，例如 `part1.sh`、`part2.sh`。入口文件只负责组织本次运行逻辑，可以调用 `Training/train.py`、`Sampling/generation.py` 或后续测评模块；具体超参数不应写死在入口文件中，也不在 `experiments/` 内部保存 config。

9.6 configs/

项目根目录下的 `configs/` 是唯一配置目录。当前配置以 Python 文件形式保存，并通过 `get_config()` 返回 `ml_collections.ConfigDict`；`configs/loader.py` 只负责加载这些配置。`configs/base_training.py` 和 `configs/base_sampling.py` 保存最基础、可复用的默认参数。具体实验配置采用与 `experiments/` 对应的层级：实验编号用文件夹表示，例如 `configs/exp1/`、`configs/exp2/`；同一实验下的不同阶段用 config 文件表示，例如 `part1_config.py`、`part2_config.py`。这些文件记录数据路径、模型规模、batch size、optimizer、训练步数、 τ 范围、loss 权重、checkpoint 路径、采样步数、eta 和 wandb 运行名等参数。

10 结论

Momentum Flow 进阶主线训练框架的核心是：通过冻结 VAE 把 RGB 图像映射为 latent z_0 ，再通过条件 kinetic Gaussian flow 构造训练样本 (z_t, v_t) ，并训练两个 MomentumMMDiT 网络 r_θ 与 $s_{\theta v}$ 。 r_θ 负责 latent 全局端点方向， $s_{\theta v}$ 负责 velocity-space 的局部分布修正。当前代码已经实现训练闭环，后续实验应在该主线大纲基础上派生，而不是把某个实验配置当作训练方案本身。